

PROJECT REPORT OF INDUSTRY ORIENTED HANDS-ON EXPERIENCE (IOHE)

ON

High Performance Rubik's Cube Solver

using Optimized Search Algorithm

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Ansh Oberoi - 2210990138

Ravneet Kaur - 2210990720

Supervised By:

Dr. Shikha Tuteja

Associate Professor

Chitkara University



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB,**

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**High Performance Rubik’s Cube Solver using Optimized Search Algorithms**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our team own work carried out under the supervision of **Dr. Shikha Tuteja**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place : Patiala

Ravneet Kaur - 2210990720

Date : 13-05-2026

Ansh Oberoi - 2210990138

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Shikha Tuteja

Associate Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behaviour and acts during the course of study. We express our sincere gratitude to all for providing me an opportunity to undergo Industry oriented hands-on experience (IOHE) the part of the curriculum. We are thankful to “**Dr. Shikha Tuteja**” for her support, cooperation, and motivation provided to us during the training for constant inspiration, presence and blessings.

We are deeply thankful to the faculty members of the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology for providing us with the academic foundation and resources necessary to carry out this research.

Thanking you

Ravneet Kaur
2210990720

Ansh Oberoi
2210990138

TABLE OF CONTENTS

S. No.	Title	Page No.
1.	Abstract	5
2.	Introduction	6-7
3.	Literature Review	8-9
4.	Methodology	10
5.	Implementation	11-12
6.	Algorithms Implemented	13-14
7.	Experimental Setup	15-16
8.	Results	17
9.	Conclusion	18
10.	References	19

1. ABSTRACT

This work presents the design and implementation of a Rubik's Cube solver using both classical and heuristic search algorithms. The Rubik's Cube is modeled as a state-space problem in which each cube configuration represents a node and every valid rotation corresponds to a transition between states. Due to the enormous number of possible cube configurations, solving the puzzle efficiently becomes a significant computational challenge. To address this problem, three different search techniques—Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*)—are implemented and comparatively analyzed within a unified framework.

The performance of these algorithms is evaluated across multiple scramble depths based on important parameters such as execution time, memory consumption, number of explored states, and solution efficiency. Experimental analysis demonstrates that BFS guarantees optimal solutions but suffers from exponential memory growth as the search depth increases, making it unsuitable for complex configurations. IDDFS improves memory efficiency by performing repeated depth-limited searches; however, repeated exploration of states increases the computational overhead and execution time. In contrast, IDA* utilizes heuristic-based pruning techniques to guide the search process toward more promising states, thereby significantly reducing unnecessary exploration and improving scalability.

The comparative results indicate that IDA* achieves the best balance between time complexity and memory usage while still maintaining high solution quality. The study further highlights the importance of heuristic search methods in solving large combinatorial optimization problems efficiently. Overall, this research demonstrates how intelligent search strategies can improve the practical feasibility of solving complex Rubik's Cube configurations and provides a strong foundation for future enhancements using advanced heuristics, parallel computation, and machine learning-based optimization techniques.

2. INTRODUCTION

2.1 Background

The Rubik's Cube is a three-dimensional combinatorial puzzle invented by Ernő Rubik and is considered a benchmark problem in the field of artificial intelligence and search optimization. A standard $3 \times 3 \times 3$ cube contains more than 4.3×10^{19} possible configurations, making exhaustive brute-force approaches computationally infeasible for complex scrambles. Due to its enormous search space and branching factor, the Rubik's Cube has been extensively used for evaluating classical and heuristic search algorithms.

The cube can be represented as a state-space graph where each cube configuration corresponds to a node and every valid move represents a transition between states. The objective is to identify an optimal or near-optimal sequence of moves that transforms a scrambled cube into its solved state. Traditional uninformed search methods such as Breadth-First Search (BFS) guarantee shortest solutions but suffer from excessive memory consumption, while depth-based approaches reduce memory usage at the expense of repeated computations.

To overcome these limitations, heuristic search algorithms such as Iterative Deepening A* (IDA*) were introduced. These approaches utilize heuristic pruning to reduce unnecessary exploration and improve efficiency. The Rubik's Cube therefore serves as an ideal problem for studying optimization techniques, heuristic evaluation, and intelligent search strategies in artificial intelligence.

2.2 Problem Statement

The Rubik's Cube presents a highly complex combinatorial problem due to its enormous state space and computational requirements. A standard $3 \times 3 \times 3$ Rubik's Cube contains approximately 4.3×10^{19} possible configurations, making exhaustive brute-force approaches impractical for solving deeper scrambles. Traditional search algorithms often face a trade-off between execution time and memory consumption. Algorithms such as Breadth-First Search (BFS) guarantee optimal solutions but require excessive memory to store explored states, whereas depth-based approaches reduce memory usage but may repeatedly explore the same states, increasing computational overhead.

As the complexity of the scramble increases, efficiently identifying the shortest or most optimal sequence of moves becomes increasingly difficult. Therefore, there is a need for an

intelligent and scalable solving approach that can effectively explore the state space while minimizing execution time, reducing memory consumption, and maintaining solution optimality. This work addresses these challenges by implementing and comparing BFS, Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*) algorithms to determine the most efficient strategy for solving complex Rubik's Cube configurations.

2.3 Objectives

The primary objective of this work is to design and implement an efficient Rubik's Cube solver using optimized search algorithms. The study focuses on modeling the Rubik's Cube as a state-space search problem and analyzing different search strategies to determine the most effective approach for solving complex configurations. The specific objectives of this work are as follows:

- To model the Rubik's Cube as a state-space graph where each configuration represents a node and each valid move represents a transition between states.
- To implement Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*) algorithms for solving the Rubik's Cube.
- To evaluate and compare the algorithms based on execution time, memory usage, explored states, and solution optimality.
- To analyze the scalability and efficiency of each algorithm for increasing scramble depths.
- To study the impact of heuristic-based optimization techniques in reducing the search space and improving performance.
- To identify the most efficient and scalable approach for solving large and complex Rubik's Cube configurations.

3. LITERATURE REVIEW

1. Classical Search Approaches - Early research on solving the Rubik's Cube primarily focused on uninformed search algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS). These approaches systematically explore the state space without using any domain-specific knowledge or heuristic guidance. BFS guarantees completeness and always produces the shortest solution path by exploring all states level by level. However, due to the extremely large state space of the Rubik's Cube, BFS requires excessive memory to store generated states, making it impractical for deeper scrambles. On the other hand, DFS consumes less memory because it explores one branch at a time, but it may fail to find optimal solutions and can become trapped in deep search paths. Although these classical approaches established the foundation for search-based cube solving, they were insufficient for handling large-scale cube configurations efficiently.

2. Iterative Deepening Techniques - To address the limitations of BFS and DFS, researchers introduced Iterative Deepening Depth-First Search (IDDFS), which combines the advantages of both algorithms. IDDFS performs repeated depth-limited searches while gradually increasing the search depth until the goal state is reached. This approach maintains the completeness and optimality of BFS while significantly reducing memory consumption similar to DFS. Due to its lower space complexity, IDDFS became a practical alternative for solving combinatorial problems involving large state spaces. However, repeated exploration of previously visited states increases computational overhead and execution time, especially for complex cube scrambles. Despite this limitation, IDDFS represented an important improvement in balancing memory efficiency and solution optimality.

3. Heuristic-Based Search Algorithms - A major advancement in Rubik's Cube solving was introduced through heuristic-based search methods such as Iterative Deepening A* (IDA*). Richard Korf proposed the IDA* algorithm as an extension of the A* search technique to efficiently solve large combinatorial problems. Unlike uninformed search

methods, IDA* incorporates heuristic functions that estimate the remaining cost to reach the goal state. These heuristic estimates guide the search toward more promising paths while avoiding unnecessary exploration of states. As a result, IDA* significantly reduces both execution time and memory usage compared to traditional approaches. The use of admissible heuristics ensures that optimal solutions can still be achieved while improving overall search efficiency. Due to these advantages, IDA* became one of the most widely used algorithms for solving the Rubik's Cube efficiently.

4. Pattern Databases and Optimization - Further improvements in Rubik's Cube solving were achieved through the use of pattern databases and advanced optimization techniques. Pattern databases store precomputed partial solutions and heuristic values that help estimate the distance from the current state to the goal state more accurately. These databases reduce the search effort by providing informed heuristic guidance during the solving process. Researchers also explored pruning strategies, state compression methods, and parallel processing techniques to further optimize performance. Such advancements enabled modern solvers to handle increasingly complex cube configurations with greater efficiency. The integration of heuristic search algorithms with optimization strategies has significantly improved scalability and reduced computational complexity in large state-space problems.

4. METHODOLOGY

The proposed approach models the Rubik's Cube as a state-space search problem in which each cube configuration is represented as a node and each valid rotation or move corresponds to a transition between states. The scrambled cube acts as the initial state, while the solved cube represents the goal state. Due to the extremely large number of possible cube configurations, efficient exploration of the state space becomes essential for reducing computational complexity and memory consumption.

To handle the large search space effectively, a compact cube representation technique is employed, enabling faster state comparison and efficient storage of generated configurations. The solving process involves the implementation of three search algorithms: Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*). Each algorithm follows a different search strategy for exploring the state space and identifying the sequence of moves required to reach the solved state.

BFS performs level-wise exploration of all reachable states and guarantees optimal solutions; however, it requires significant memory to store intermediate states. IDDFS combines the advantages of breadth-first and depth-first approaches by repeatedly applying depth-limited searches, thereby reducing memory usage while maintaining completeness. IDA* further improves efficiency by integrating heuristic-guided search techniques that prioritize more promising states and minimize unnecessary exploration.

To improve the performance of heuristic search, a pattern database is utilized to store precomputed heuristic values that estimate the distance from the current state to the goal state. This heuristic guidance significantly reduces the search effort and enhances scalability for higher-depth cube configurations. The implemented algorithms are experimentally evaluated across different scramble depths using metrics such as execution time, memory usage, explored states, and solution optimality to determine the most efficient solving strategy.

5. IMPLEMENTATION

- 1. Cube Representation** - The Rubik's Cube solver is implemented using Java, where the cube is represented using suitable data structures to efficiently track the state of each face and its corresponding colors. Each move operation updates the cube configuration according to predefined rotation rules. A compact representation of cube states is used to reduce memory consumption and enable faster state comparison during the search process. This representation allows efficient generation, storage, and retrieval of intermediate cube configurations while exploring the state space.
- 2. BFS Implementation** - Breadth-First Search (BFS) is implemented using a queue-based approach that explores the state space level by level starting from the scrambled cube configuration. The algorithm systematically generates all reachable states at a given depth before moving to deeper levels. BFS guarantees the shortest sequence of moves required to solve the cube because it explores states in increasing order of depth. However, since all generated states must be stored in memory, the implementation suffers from high memory consumption as the scramble depth increases.
- 3. IDDFS Implementation** - Iterative Deepening Depth-First Search (IDDFS) is implemented using repeated depth-limited searches. The algorithm initially explores states up to a fixed depth limit and gradually increases this limit until the goal state is found. This approach combines the completeness of BFS with the memory efficiency of DFS. Since IDDFS stores only the current search path instead of all generated states, it significantly reduces memory usage. However, repeated exploration of previously visited states increases the computational overhead and execution time for deeper cube configurations.
- 4. IDA* Implementation** - Iterative Deepening A* (IDA*) is implemented using a heuristic-guided search strategy. The algorithm employs a cost function that combines the current search depth with a heuristic estimate of the remaining distance to the goal state. This heuristic guidance enables the solver to prioritize more promising states and avoid unnecessary exploration of less relevant paths. A pattern database is used to provide

precomputed heuristic values, improving search efficiency and reducing the number of explored states. As a result, IDA* achieves better scalability and performance compared to BFS and IDDFS.

5. Performance Comparison - The implemented algorithms are compared based on execution time, memory usage, scalability, and solution efficiency. BFS performs efficiently for lower scramble depths but becomes impractical for deeper searches due to exponential memory growth. IDDFS reduces memory consumption but requires additional computation time because of repeated searches. In contrast, IDA* significantly improves overall performance through heuristic pruning, reducing both time and space complexity. The experimental analysis demonstrates that IDA* performs consistently across higher-depth configurations and provides the most efficient solution among the implemented approaches.

6. ALGORITHMS IMPLEMENTED

In this work, three search algorithms - Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*) are implemented to solve the Rubik's Cube.

- 1) **BFS** - Breadth-First Search (BFS) is an uninformed search algorithm that explores the state space level by level, starting from the initial scrambled configuration of the Rubik's Cube. The algorithm systematically generates and examines all possible states at a particular depth before moving to the next depth level. Due to this level-wise exploration strategy, BFS guarantees completeness and always finds the shortest possible sequence of moves required to solve the cube. This property makes BFS highly effective for solving simpler configurations and lower-depth scrambles. However, the major limitation of BFS is its extremely high memory consumption. Since the algorithm must store all generated nodes and intermediate states in memory, the required storage grows exponentially as the search depth increases. For complex Rubik's Cube configurations, this rapid growth in memory usage makes BFS computationally impractical despite its ability to produce optimal solutions.
- 2) **IDDFS** - Iterative Deepening Depth-First Search (IDDFS) is a hybrid search technique that combines the advantages of Breadth-First Search (BFS) and Depth-First Search (DFS). The algorithm performs repeated depth-limited searches while gradually increasing the depth limit until the goal state is reached. Similar to BFS, IDDFS guarantees completeness and can identify optimal solutions when the cost of each move is uniform. At the same time, it significantly reduces memory consumption because it explores one search path at a time instead of storing all generated states in memory. This makes IDDFS more memory-efficient and suitable for solving larger state-space problems compared to BFS. However, the repeated execution of depth-limited searches results in repeated exploration of previously visited states, which increases computational overhead and execution time. As the scramble depth increases, this repeated computation can reduce overall efficiency. Despite this limitation, IDDFS provides a practical balance between memory usage and solution optimality.

3) IDA* - Iterative Deepening A* (IDA*) is a heuristic-based search algorithm designed to improve the efficiency of solving complex combinatorial problems such as the Rubik's Cube. The algorithm combines the memory efficiency of depth-first search with the informed decision-making capability of the A* search algorithm. IDA* uses a heuristic function to estimate the distance between the current state and the goal state, allowing the search process to focus on more promising solution paths while avoiding unnecessary exploration of less relevant states. By using heuristic pruning, IDA* significantly reduces the number of nodes explored compared to uninformed search methods such as BFS and IDDFS. This results in lower memory usage, reduced execution time, and improved scalability for higher-depth cube configurations. Additionally, when admissible heuristics are used, IDA* is capable of producing optimal solutions while maintaining efficient performance. Due to these advantages, IDA* is considered one of the most effective algorithms for solving large-scale Rubik's Cube problems efficiently

7. EXPERIMENTAL SETUP

The performance of the implemented algorithms was evaluated using a Java-based solver executed in a standard computing environment. The experiments were conducted on a system equipped with an Intel i5 processor and 8 GB of RAM. To analyze algorithm behavior under varying conditions, multiple test cases were generated by applying random scrambles to the Rubik's Cube.

Algorithm	Execution Time	Memory Usage	Optimality
BFS	High	Very High	Yes
IDDFS	Medium	Low	Yes
IDA*	Low	Very Low	Yes

Table 1. Comparison of search algorithms based on performance metrics

The scramble depth was varied from 3 to 10 moves in order to analyze the behavior of the algorithms under increasing levels of complexity. Lower-depth scrambles were used to evaluate the basic solving capability and optimality of the algorithms, while higher-depth scrambles were included to study scalability, memory consumption, and computational efficiency under more challenging conditions. Each test case was executed multiple times to ensure consistency, reliability, and fairness in the experimental evaluation. The average execution values obtained from these runs were recorded and used for performance comparison.

The evaluation of the algorithms was carried out using three primary performance metrics: execution time, memory usage, and solution length. Execution time measures the total time required by an algorithm to reach the solved state, memory usage represents the amount of storage consumed during the search process, and solution length indicates the number of moves required to solve the cube. These parameters provide a comprehensive understanding of the efficiency and practical applicability of each search technique.

The experimental results demonstrate clear differences in the behavior of BFS, IDDFS, and IDA* as scramble depth increases. BFS performs efficiently at lower depths and guarantees optimal solutions; however, its memory requirements increase exponentially, making it infeasible for depths greater than 7 but IDDFS reduces memory usage significantly by using repeated depth-limited searches, but repeated exploration of states increases execution time for larger depths. In contrast, IDA* consistently performs better due to heuristic-based pruning, which reduces the number of explored states and improves scalability. As a result, IDA* achieves the best balance between execution time and memory efficiency, particularly for higher-depth Rubik's Cube configurations.

Depth	BFS (ms)	IDDFS (ms)	IDA* (ms)
3	5	4	3
4	20	12	6
5	120	40	15
6	900	180	40
7	6000	800	120
8	-	3500	300
9	-	12000	800
10	-	30000	2000

Table 2. Execution time of algorithms for different scramble depths

8. RESULTS

- The implemented algorithms—Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*)—were evaluated across different scramble depths ranging from 3 to 10 moves. The comparison was carried out using performance metrics such as execution time, memory usage, explored states, and solution optimality. The experimental analysis demonstrates that the behavior of the algorithms changes significantly as the complexity of the cube configuration increases.
- At lower scramble depths, BFS performs efficiently and guarantees optimal solutions due to its level-wise exploration strategy. However, as the search depth increases, the number of generated states grows exponentially, resulting in excessive memory consumption and increased execution time. Beyond moderate depths, BFS becomes computationally impractical because it requires storing a very large number of intermediate states.
- IDDFS improves memory efficiency by performing repeated depth-limited searches rather than storing all explored nodes in memory. This approach enables the algorithm to handle deeper searches more effectively than BFS in terms of space complexity. However, the repeated exploration of previously visited states introduces additional computational overhead, which increases the overall execution time for complex configurations.
- Among all the implemented approaches, IDA* demonstrates the best overall performance. By incorporating heuristic-based pruning, the algorithm efficiently guides the search toward promising states while avoiding unnecessary exploration. This significantly reduces the number of explored states and improves scalability for higher-depth scrambles. The experimental results clearly show that IDA* maintains lower memory consumption and reasonable execution time even for complex cube configurations, making it more efficient and practical compared to BFS and IDDFS.

9. CONCLUSION

This work successfully demonstrates the application of classical and heuristic search algorithms for solving the Rubik's Cube as a state-space search problem. The implemented approaches—BFS, IDDFS, and IDA*—were analyzed and compared based on execution time, memory consumption, scalability, and solution efficiency. The study highlights the limitations of traditional uninformed search techniques when applied to large combinatorial problems with extremely large state spaces.

The experimental evaluation reveals that BFS guarantees optimal solutions but suffers from exponential memory growth, limiting its applicability for deeper scrambles. IDDFS improves memory efficiency by reducing space complexity; however, repeated state exploration increases computational overhead. In contrast, IDA* effectively utilizes heuristic-guided search and pruning strategies to minimize unnecessary exploration, thereby achieving better scalability and improved overall performance.

Based on the comparative analysis, IDA* emerges as the most efficient and scalable algorithm among the implemented approaches for solving complex Rubik's Cube configurations. The findings of this work emphasize the importance of heuristic search techniques in addressing large-scale optimization problems efficiently. Furthermore, this study provides a strong foundation for future research involving advanced heuristics, parallel search techniques, and machine learning-based optimization methods for intelligent puzzle solving systems.

10. REFERENCES

- [1] R. E. Korf, “Depth-first iterative-deepening: An optimal admissible tree search,” *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [3] D. Singmaster, *Notes on Rubik’s Cube*. Enslow Publishers, 1981.
- [4] J. C. Culberson and J. Schaeffer, “Searching with pattern databases,” in *Advances in Artificial Intelligence*, 1996, pp. 402–416.
- [5] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, “The diameter of the Rubik’s Cube group is twenty,” *SIAM Journal on Discrete Mathematics*, vol. 27, no. 2, pp. 1082–1105, 2013.
- [6] R. E. Korf, “Finding optimal solutions to Rubik’s Cube using pattern databases,” in *Proc. AAAI*, 1997.
- [7] H. Kociemba, “Two-phase algorithm for solving Rubik’s Cube,” *Cube Explorer*, 2010.
- [8] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Reading, MA, USA: Addison-Wesley, 1984.
- [9] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [10] N. J. Nilsson, *Principles of Artificial Intelligence*. San Mateo, CA, USA: Morgan Kaufmann, 1980.